

Name :- Mihir Mungara

Snowflake Assignment-9 :-

Profile and Optimize Queries Using Snowflake's Query Profiler

Showing Customer Table:-

```
-- creating tables
-- customer table
CREATE OR REPLACE TABLE CUSTOMERS (
  CUSTOMER_ID INT,
  CUSTOMER_NAME STRING,
  CITY STRING
);

INSERT INTO CUSTOMERS
SELECT
  SEQ4()+1,
  'Customer_' || (SEQ4()+1),
  CASE MOD(SEQ4(),5)
    WHEN 0 THEN 'New York'
    WHEN 1 THEN 'Chicago'
    WHEN 2 THEN 'Dallas'
    WHEN 3 THEN 'Boston'
    ELSE 'Seattle'
  END
FROM TABLE(GENERATOR(ROWCOUNT => 100000));
```

Showing Sample Data :-

46 select * from customers limit 10;

Results (just now)

Table Chart Pivot

10 rows 92ms

	CUSTOMER_ID	CUSTOMER_NAME	CITY
1	1	Customer_1	New York
2	2	Customer_2	Chicago
3	3	Customer_3	Dallas
4	4	Customer_4	Boston
5	5	Customer_5	Seattle
6	6	Customer_6	New York
7	7	Customer_7	Chicago
8	8	Customer_8	Dallas
9	9	Customer_9	Boston
10	10	Customer_10	Seattle

Showing Order Table :-

```
-- order table
CREATE OR REPLACE TABLE ORDERS (
  ORDER_ID INT,
  CUSTOMER_ID INT,
  ORDER_AMOUNT NUMBER(10,2),
  ORDER_DATE DATE
);

INSERT INTO ORDERS
SELECT
  SEQ4()+1,
  UNIFORM(1,100000,RANDOM()),
  UNIFORM(100,5000,RANDOM()),
  DATEADD(
    DAY,
    -UNIFORM(1,365,RANDOM()),
    CURRENT_DATE()
  )
FROM TABLE(GENERATOR(ROWCOUNT => 1000000));
```

Showing Sample Data :-

46  select * from orders limit 10;

Results (just now)

Table Chart Pivot

🔍 🏠 10 rows 67ms 📄 ⬇️ ⬆️ ⌚

	ORDER_ID	CUSTOMER_ID	ORDER_AMOUNT	ORDER_DATE
1	1	44984	1715.00	2026-03-05
2	2	6674	4032.00	2026-04-22
3	3	63584	2993.00	2025-09-04
4	4	96813	2534.00	2026-03-14
5	5	83041	4489.00	2026-05-09
6	6	23242	1726.00	2025-11-14
7	7	93759	712.00	2026-04-01
8	8	22476	399.00	2025-06-27
9	9	87551	2418.00	2026-05-09
10	10	86179	810.00	2025-06-22

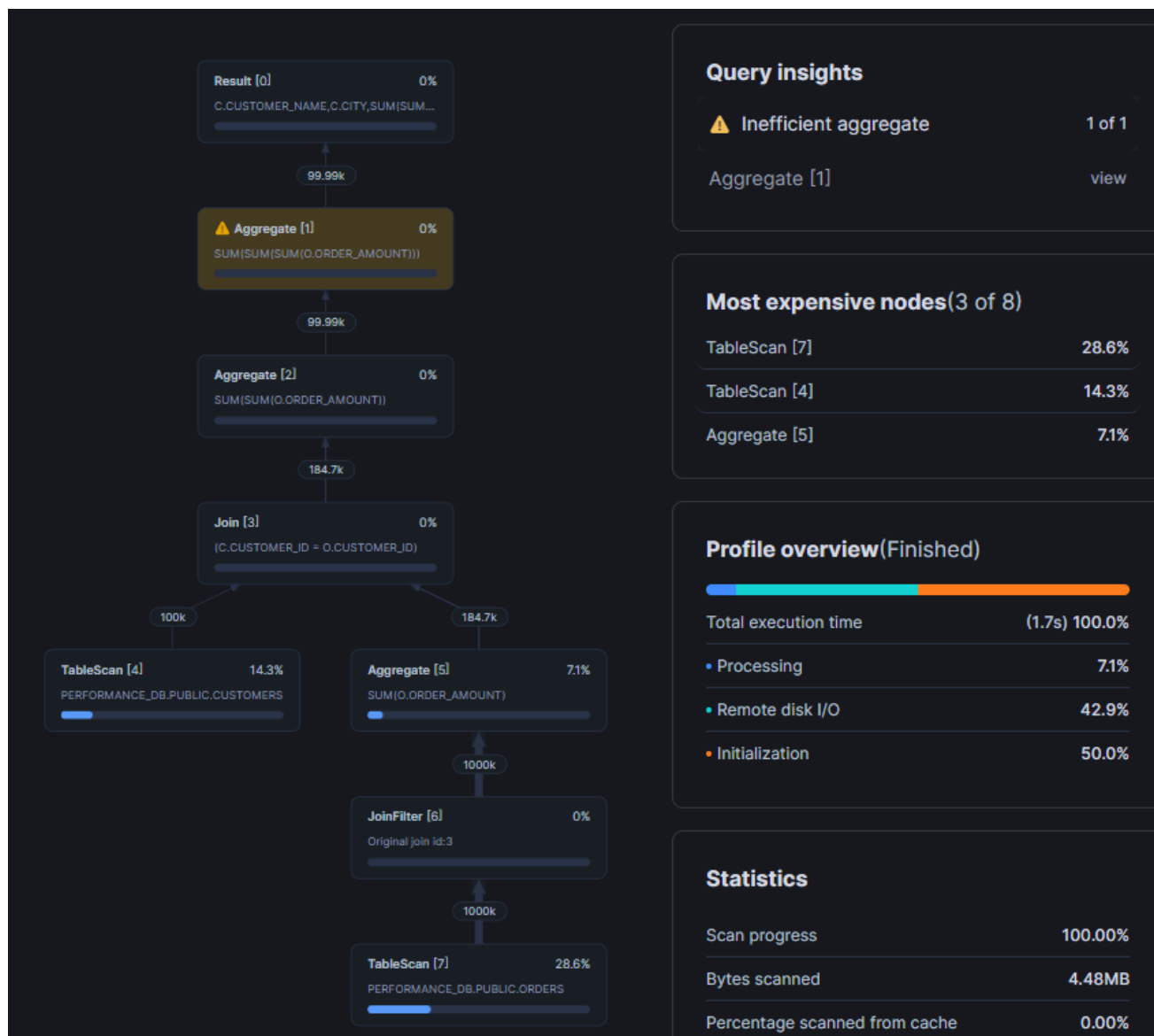
Analyzing the Profile

- Look for bottlenecks such as high scan percentages or large intermediate results.

Query :-

```
select * from orders limit 10;
select * from customers limit 10;
SELECT COUNT(*) FROM CUSTOMERS;
SELECT COUNT(*) FROM ORDERS;

SELECT
  c.CUSTOMER_NAME,
  c.CITY,
  SUM(o.ORDER_AMOUNT) AS TOTAL_SALES
FROM CUSTOMERS c
JOIN ORDERS o
  ON c.CUSTOMER_ID = o.CUSTOMER_ID
GROUP BY
  c.CUSTOMER_NAME,
  c.CITY;
```



- Optimize queries by:

□ Adding filters (e.g., WHERE clauses).

```
SELECT
    c.CUSTOMER_NAME,
    c.CITY,
    SUM(o.ORDER_AMOUNT)
FROM CUSTOMERS c
JOIN ORDERS o
ON c.CUSTOMER_ID=o.CUSTOMER_ID
WHERE o.ORDER_DATE >= DATEADD(DAY,-30,CURRENT_DATE())
GROUP BY
    c.CUSTOMER_NAME,
    c.CITY;
```



Query insights

⚠ Inefficient aggregate 1 of 1

Aggregate [1] [view](#)

Most expensive nodes(2 of 10)

TableScan [6]	45.5%
TableScan [9]	18.2%

Profile overview(Finished)

Total execution time (1.4s) 100.0%

- Remote disk I/O 63.6%
- Initialization 36.4%

Statistics

Scan progress	100.00%
Bytes scanned	5.43MB
Percentage scanned from cache	0.00%
Bytes written to result	0.33MB
Partitions scanned	2

❑ Using proper indexes or clustering keys.

```
ALTER TABLE ORDERS
CLUSTER BY (ORDER_DATE);

SHOW TABLES LIKE 'ORDERS';
```

```
graph BT
    TS6[TableScan [6] 20%] -- 82.33k --> F5[Filter [5] 0%]
    TS9[TableScan [9] 10%] -- 56.36k --> JF8[JoinFilter [8] 0%]
    F5 -- 82.33k --> A4[Aggregate [4] 0%]
    JF8 -- 56.36k --> A7[Aggregate [7] 10%]
    A4 -- 82.33k --> J3[Join [3] 0%]
    A7 -- 56.36k --> J3
    J3 -- 82.33k --> A2[Aggregate [2] 0%]
    A2 -- 56.26k --> A1[Aggregate [1] 0%]
    A1 -- 56.26k --> R0[Result [0] 10%]
```

Query insights

⚠ Inefficient aggregate

1 of 1

Aggregate [1]

view

Most expensive nodes(4 of 10)

TableScan [6]	20.0%
TableScan [9]	10.0%
Aggregate [7]	10.0%
Result [0]	10.0%

Profile overview(Finished)

Total execution time (1.2s) 100.0%

Processing 10.0%

Remote disk I/O 40.0%

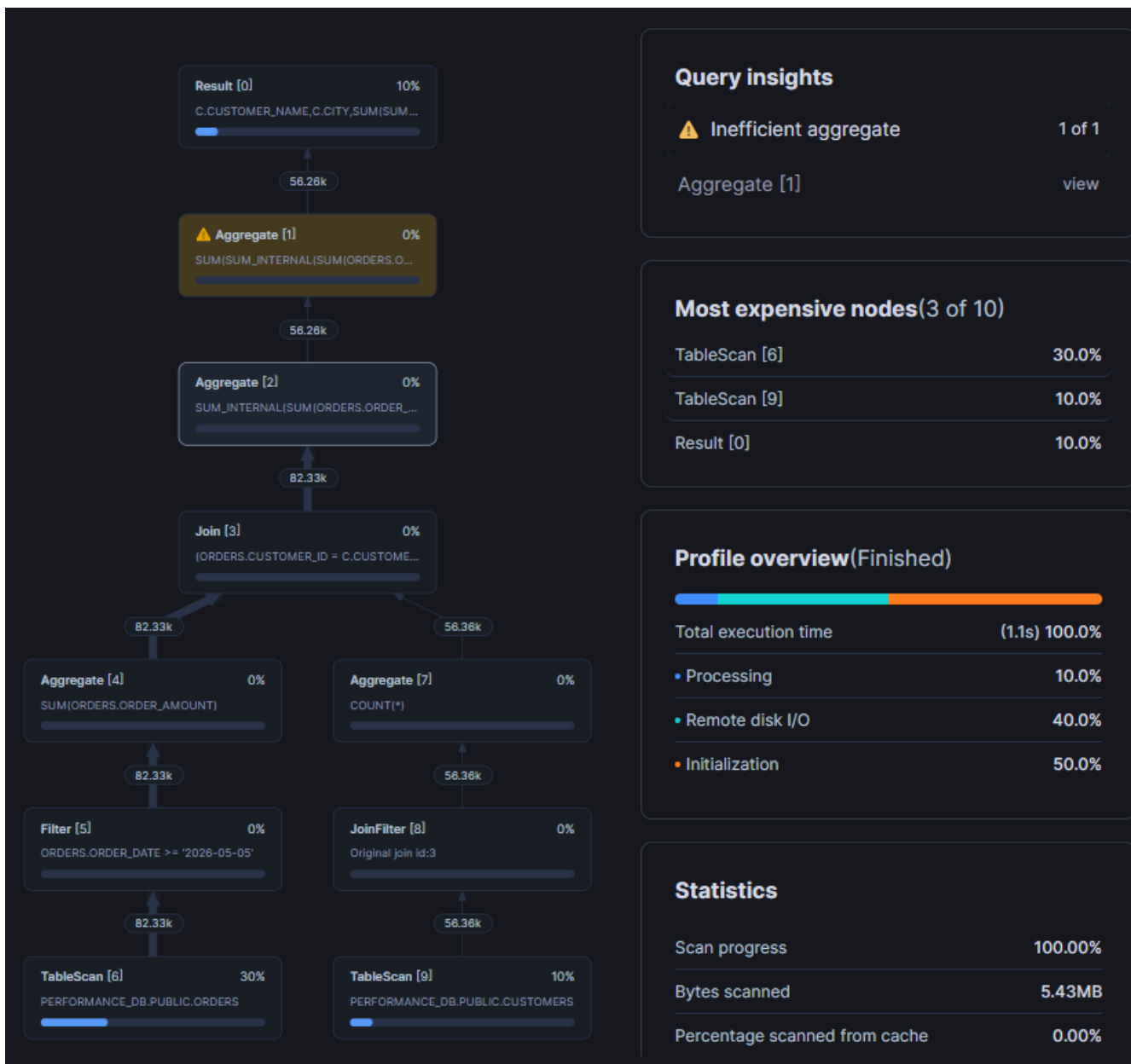
Initialization 50.0%

Statistics

Scan progress	100.00%
Bytes scanned	5.43MB

❑ Minimizing unnecessary joins or subqueries.

```
WITH FILTERED_ORDERS AS
(
    SELECT *
    FROM ORDERS
    WHERE ORDER_DATE >= DATEADD(DAY, -30, CURRENT_DATE())
)
SELECT
    c.CUSTOMER_NAME,
    c.CITY,
    SUM(f.ORDER_AMOUNT)
FROM CUSTOMERS c
JOIN FILTERED_ORDERS f
ON c.CUSTOMER_ID=f.CUSTOMER_ID
GROUP BY
    c.CUSTOMER_NAME,
    c.CITY;
```



2. Experiment with Clustering Keys for Large Tables

Step 1: Create a Large Dataset

- Load a sample dataset into a table.

Step 2: Add a Clustering Key

- Choose a column frequently used in WHERE clauses.
- Add a clustering key using:

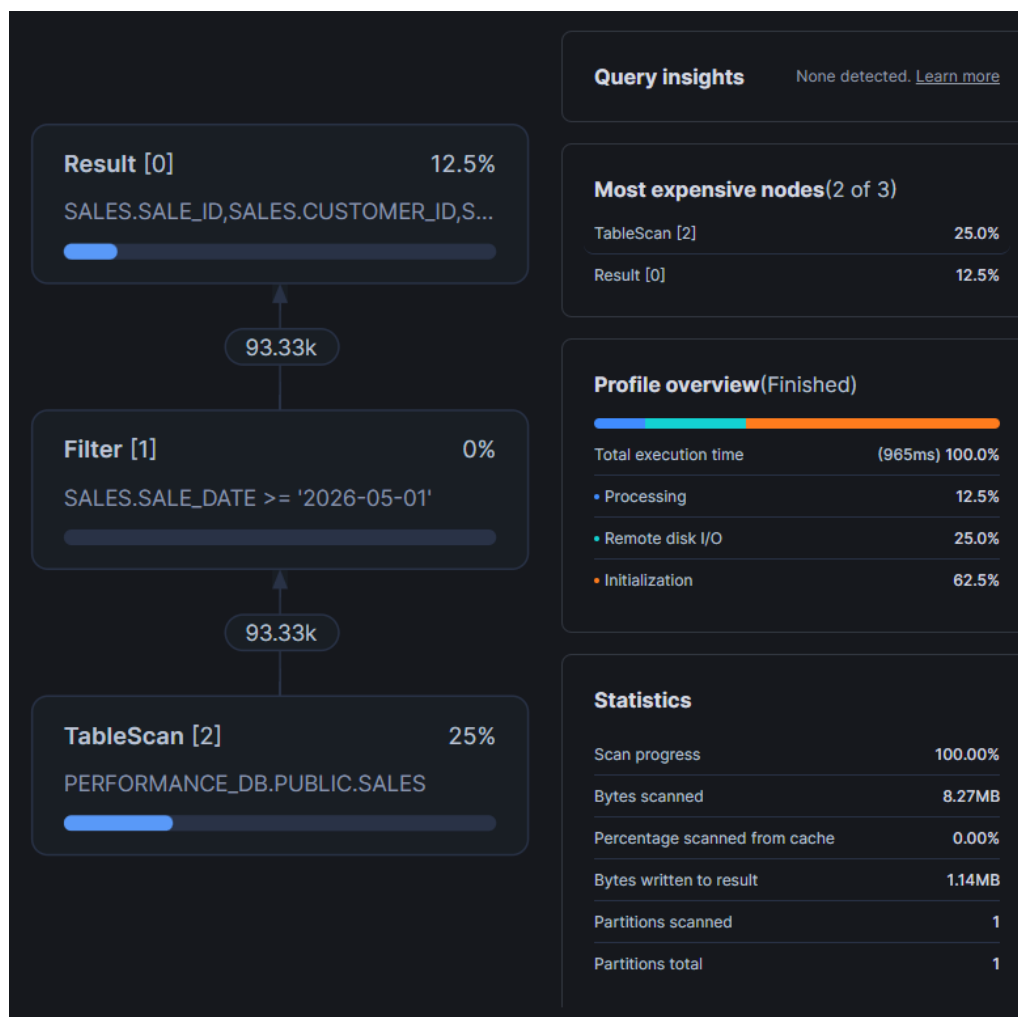
```
ALTER TABLE sales ADD CLUSTERING KEY (sale_date);
```

Step 3: Monitor Clustering Performance

- Check clustering depth:

```
SELECT SYSTEM$CLUSTERING_INFORMATION('sales');
```

Without Clustering :-



With Clustering :-



```
{
  "cluster_by_keys" : "LINEAR(SALE_DATE)",
  "notes" : "Clustering key columns contain high cardinality key SALE_DATE which might result in expensive re-clustering. Consider reducing the cardinality of clustering keys. Please refer to https://docs.snowflake.net/manuals/user-guide/tables-clustering-keys.html for more information.",
  "total_partition_count" : 1,
  "total_constant_partition_count" : 0,
  "average_overlaps" : 0.0,
  "average_depth" : 1.0,
  "partition_depth_histogram" : {
    "00000" : 0,
    "00001" : 1,
    "00002" : 0,
    "00003" : 0,
    "00004" : 0,
    "00005" : 0,
    "00006" : 0,
    "00007" : 0,
    "00008" : 0,
    "00009" : 0,
    "00010" : 0,
    "00011" : 0,
    "00012" : 0,
    "00013" : 0,
    "00014" : 0,
    "00015" : 0,
    "00016" : 0
  },
  "clustering_errors" : [ ]
}
```

3. Monitor and Analyze Cost Usage Metrics

Checked Warehouse Credit Usage :-

```
SELECT
    WAREHOUSE_NAME,
    SUM(CREDITS_USED) AS TOTAL_CREDITS
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME > CURRENT_TIMESTAMP() - INTERVAL '7 DAY'
GROUP BY WAREHOUSE_NAME
ORDER BY TOTAL_CREDITS DESC;
```

Results (just now)

TableChartPivot

	WAREHOUSE_NAME	TOTAL_CREDITS
1	COMPUTE_WH	6.045369456
2	TEST_WAREHOUSE	1.175645003
3	PUBLIC_WH	0.120532778
4	MY_WAREHOUSE	0.109892222
5	CLOUD_SERVICES_ONLY	0.000698054

View Detailed Warehouse Usage :-

```
SELECT
    WAREHOUSE_NAME,
    START_TIME,
    END_TIME,
    CREDITS_USED
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
ORDER BY START_TIME DESC;
```

Results (just now)

TableChartPivot

76 rows2.4s

	WAREHOUSE_NAME	START_TIME	END_TIME	CREDITS_USED
1	COMPUTE_WH	2026-06-04 22:00:00.000 -0700	2026-06-04 23:00:00.000 -0700	0.322898056
2	COMPUTE_WH	2026-06-04 21:00:00.000 -0700	2026-06-04 22:00:00.000 -0700	0.093745833
3	COMPUTE_WH	2026-06-04 05:00:00.000 -0700	2026-06-04 06:00:00.000 -0700	0.080738887
4	COMPUTE_WH	2026-06-04 05:00:00.000 -0700	2026-06-04 06:00:00.000 -0700	0.051093886
5	COMPUTE_WH	2026-06-04 04:00:00.000 -0700	2026-06-04 05:00:00.000 -0700	0.116846389
6	COMPUTE_WH	2026-06-04 04:00:00.000 -0700	2026-06-04 05:00:00.000 -0700	0.038024168
7	COMPUTE_WH	2026-06-04 03:00:00.000 -0700	2026-06-04 04:00:00.000 -0700	0.117395000
8	TEST_WAREHOUSE	2026-06-04 03:00:00.000 -0700	2026-06-04 04:00:00.000 -0700	0.114048614
9	COMPUTE_WH	2026-06-04 02:00:00.000 -0700	2026-06-04 03:00:00.000 -0700	0.221807223
10	TEST_WAREHOUSE	2026-06-04 02:00:00.000 -0700	2026-06-04 03:00:00.000 -0700	0.263969167
11	COMPUTE_WH	2026-06-04 01:00:00.000 -0700	2026-06-04 02:00:00.000 -0700	0.000082221
12	TEST_WAREHOUSE	2026-06-04 01:00:00.000 -0700	2026-06-04 02:00:00.000 -0700	0.123337499

Check Query Cost Contributors :-

```
SELECT
    QUERY_ID,
    USER_NAME,
    WAREHOUSE_NAME,
    EXECUTION_TIME
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
ORDER BY START_TIME DESC
LIMIT 20;|
```

Results (just now)

TableChartPivot

20 rows5.9s

	QUERY_ID	USER_NAME	WAREHOUSE_NAME	EXECUTION_TIME
1	01c4d6fb-0001-ea65-001c-9f4b0014856e	MIHIR	COMPUTE_WH	1
2	01c4d6fa-0001-ebb8-001c-9f4b001584d6	MIHIR	COMPUTE_WH	2
3	01c4d6fa-0001-ea65-001c-9f4b00148566	MIHIR	COMPUTE_WH	2
4	01c4d6f9-0001-ea65-001c-9f4b00148562	MIHIR	COMPUTE_WH	21
5	01c4d6f9-0001-ebb8-001c-9f4b001584d2	MIHIR	COMPUTE_WH	8
6	01c4d6f9-0001-ea65-001c-9f4b0014855e	MIHIR	COMPUTE_WH	2
7	01c4d6f9-0001-eb77-001c-9f4b001564fa	MIHIR	COMPUTE_WH	138
8	01c4d6f9-0001-ebb8-001c-9f4b001584ce	MIHIR	COMPUTE_WH	1
9	01c4d6f9-0001-ebb8-001c-9f4b001584ca	MIHIR	COMPUTE_WH	2
10	01c4d6f9-0001-eb77-001c-9f4b001564f6	MIHIR	COMPUTE_WH	1
11	01c4d6f9-0001-ea65-001c-9f4b0014855a	MIHIR	COMPUTE_WH	1
12	01c4d6f9-0001-ea65-001c-9f4b00148556	MIHIR	COMPUTE_WH	1
13	01c4d6f9-0001-ea65-001c-9f4b00148552	MIHIR	COMPUTE_WH	9
14	01c4d6f9-0001-ebb8-001c-9f4b001584c6	MIHIR	COMPUTE_WH	54
15	01c4d6f9-0001-eb77-001c-9f4b001564f2	MIHIR	COMPUTE_WH	2
16	01c4d6f9-0001-ea65-001c-9f4b0014854e	MIHIR	COMPUTE_WH	2